

Reviewer Pack — Kendall tau Distance and Resolution Proof Length

Entry 0a9a0b3ad00c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 16:37:04 UTC

Kendall tau Distance and Resolution Proof Length

Entry ID: 0a9a0b3ad00c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 16:37:04 UTC

1. Conjecture

Field A (mathematical branch): Statistical Measures of Agreement **Field B** (complexity object): Resolution Proof Complexity

Statement:

The Kendall tau distance between the literal frequencies in a resolution proof and the uniform distribution is $\Theta(n^{0.5})$ for all 3-CNF instances with at most 40 variables.

Rationale (proposer's reasoning):

Kendall tau distance measures the degree of agreement between two rankings, which can be used to quantify the similarity between the literal frequencies in a resolution proof and an idealized distribution. This conjecture posits that the higher the Kendall tau distance, the more non-random or complex the structure of the proof.

Taxonomy category: AGREEMENT_MEASURES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3e38bcd3bc1b3f07

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean Kendall tau distance across all seeds for the literal frequencies in resolution proofs from random 3-CNF instances with $n \leq 40$ variables is within ± 0.5 of $n^{0.5}$, and no seed produces a Kendall tau distance greater than $n^{0.5} + 1$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - (Kendall tau AND distance) AND (resolution OR proof length) - (statistical measures OF agreement) AND (proof complexity OR resolution proof) - ($\theta(n^{0.5})$ AND uniform distribution) AND (resolution proof OR proof length)

Top relevant hits considered: - [http://arxiv.org/abs/0910.5115v3] Geodesics and flows in a Poissonian city - [http://arxiv.org/abs/1810.06234v2] On kernel-based estimation of conditional Kendall's tau: finite-distance bounds and asymptotic behavior - [http://arxiv.org/abs/math/0701140v2] Short-length routes in low-cost networks via Poisson line patterns - [http://arxiv.org/abs/1511.06778v1] The 1st Fermi Lat Supernova Remnant Catalog - [http://arxiv.org/abs/1606.05050v1] Proof Complexity Lower Bounds from Algebraic Circuit Complexity - [http://arxiv.org/abs/2209.11347v2] A second moment proof of the spread lemma - [http://arxiv.org/abs/1911.10590v2] A proof of the Conjecture of Lehmer - [http://arxiv.org/abs/cs/0107004v1] On Concurrent and Resettable Zero-Knowledge Proofs for NP

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def random_3cnf(n, m):
    clauses = []
    for _ in range(m):
        literals = random.sample(range(1, n + 1), 3)
        clause = [(random.choice([-1, 1]) * lit) for lit in literals]
        clauses.append(clause)
    return clauses

def dpll(clauses, assignment, literals):
    unit_clauses = [c for c in clauses if len(c) == 1]
    while unit_clauses:
        literal = unit_clauses[0][0]
        polarity = literal > 0
        assignment[literal] = polarity
        literals.remove(literal)
        new_clauses = []
        for clause in clauses:
            if not any(abs(lit) == abs(literal) for lit in clause):
                new_clauses.append(clause)
            elif all(abs(lit) != abs(literal) for lit in clause):
                return False
        clauses = new_clauses
```

```

    unit_clauses = [c for c in clauses if len(c) == 1]
    pure_literals = []
    for literal in literals:
        if literal not in assignment and -literal not in assignment:
            count_pos = sum(1 for c in clauses if literal in c)
            count_neg = sum(1 for c in clauses if -literal in c)
            if count_pos > count_neg:
                pure_literals.append((literal, True))
            elif count_neg > count_pos:
                pure_literals.append((-literal, False))
    while pure_literals:
        literal, polarity = pure_literals[0]
        assignment[literal] = polarity
        literals.remove(literal)
        new_clauses = []
        for clause in clauses:
            if not any(abs(lit) == abs(literal) for lit in clause):
                new_clauses.append(clause)
            elif all(abs(lit) != abs(literal) for lit in clause):
                return False
        clauses = new_clauses
    return True

def resolution_proof(clauses):
    assignment = {}
    literals = set()
    for clause in clauses:
        for literal in clause:
            literals.add(abs(literal))
    while not dpll(clauses, assignment, literals):
        unit_clause = next((c for c in clauses if len(c) == 1), None)
        if unit_clause is None:
            return []
        literal = unit_clause[0]
        polarity = literal > 0
        assignment[literal] = polarity
        literals.remove(literal)
        new_clauses = []
        for clause in clauses:
            if not any(abs(lit) == abs(literal) for lit in clause):
                new_clauses.append(clause)
            elif all(abs(lit) != abs(literal) for lit in clause):
                return []
        clauses = new_clauses
    return []

def kendall_tau_distance(freqs, n):
    rank = {lit: i + 1 for i, lit in enumerate(sorted(freqs))}
    tau_numerator = sum((rank[lit] - (i + 1)) ** 2 for i, lit in enumerate(freqs))
    tau_denominator = n * (n**2 - 1) / 4
    return math.sqrt(tau_numerator / tau_denominator)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for n in [5, 10, 15, 20, 30, 40]:

```

```

    m = 2 * n
    clauses = random_3cnf(n, m)
    proof = resolution_proof(clauses)
    if not proof:
        continue
    literal_freqs = {lit: sum(1 for clause in proof if lit in clause) for lit in range(1, n + 1)}
    tau_distance = kendall_tau_distance(literal_freqs, n)
    results.append(tau_distance)
mean_value = sum(results) / len(results)
conjecture_holds = all(abs(tau - math.sqrt(n)) <= 0.5 for tau in results) and all(tau <= math.sqrt(n))
return {
    "metric_name": "Kendall_tau_distance",
    "metric_value": mean_value,
    "instances_tested": len(results),
    "n_max": max([5, 10, 15, 20, 30, 40]),
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else "mapping_undefined"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result["metric_value"])
    mean_value = sum(results) / len(results)
    support_fraction = sum(1 for r in results if abs(r - math.sqrt(n)) <= 0.5 for n in [5, 10, 15, 20, 30, 40]) / len(results)
    if all(abs(r - math.sqrt(n)) <= 0.5 for r in results) and all(r <= math.sqrt(n) + 1 for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}")
    elif any(r > math.sqrt(n) + 1 for r, n in zip(results, [5, 10, 15, 20, 30, 40])):
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={seeds[results.index(max(results))]}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9cd3ald9.py", line 120, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9cd3ald9.py", line 98, in run_trial
    proof = resolution_proof(clauses)
            ^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9cd3ald9.py", line 69, in resolution_proof
    while not dpll(clauses, assignment, literals):
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9cd3ald9.py", line 53, in dpll
    literals.remove(literal)

```

KeyError: -1

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating whether the conjecture is supported or falsified. | next: Investigate and fix the crash in the test code to proceed with the evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14634
2	preregistration	ollama_remote	glm4:latest	0	0	13468
3	novelty	ollama_remote	glm4:latest	0	0	8442
4	novelty	ollama_remote	glm4:latest	0	0	10908
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14763
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	29709
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22117
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27767
9	judge	ollama_remote	glm4:latest	0	0	8887

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 150695 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0a9a0b3ad00c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0a9a0b3ad00c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0a9a0b3ad00c.tar.gz` (if generated)